EXPERIMENT- 6

Aim: To draw the behavioral view diagram for <case study name>: Sequence diagram

Theory:

What is Behavioral modeling?

All behavioural models really do is describe the control structure of a system. This can be things like:

- Sequence of operations
- Object states
- and Object interactions

Furthermore, this modelling layer can also be called Dynamic Modelling. The activity of creating a behavioural model is commonly known as behavioural modelling. As well as this, a system should also only have one behavioural model – much like functional modelling.

Two types of behavioral model are:

• Data processing models that show how data is processed as it moves through the system;

• State machine models that show the systems response to events.

• These models show different perspectives so both of them are required to describe the system's behavior.

How to draw Sequence diagrams?

A **sequence diagram** is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called **event diagrams**, **event scenarios**, and **timing diagrams**.A sequence diagram shows, as parallel vertical lines (*lifelines*), different processes or objects that live simultaneously, and, as horizontal

arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

If the lifeline is that of an object, it demonstrates a role. Note that leaving the instance name blank can represent anonymous and unnamed instances.

Messages, written with horizontal arrows with the message name written above them, display interaction. Solid arrows with full heads are asynchronous calls and dashed arrows with stick heads are return messages. This definition is true as of UML 2, considerably different from UML 1.x. a caller sends a synchronous message, it must wait until the message is done, such as invoking a subroutine. If a caller sends an asynchronous message, it can continue processing and doesn't have to wait for a response. You see asynchronous calls in multithreaded applications and in message-oriented middleware. Activation boxes, or method-call boxes, are opaque rectangles drawn on top of lifelines to represent that processes are being performed in response to the message (ExecutionSpecifications in UML).

Objects calling methods on themselves use messages and add new activation boxes on top of any others to indicate a further level of processing.When an object is destroyed (removed from memory), an X is drawn on top of the lifeline, and the dashed line ceases to be drawn below it (this is not the case in the first example though). It should be the result of a message, either from the object itself, or another.

Procedure:

1) Identify various elements such as controller class, objects, boundaries, messages etc. of the sequence diagram.
2) Draw the sequence diagram as per the guidelines.
3) Identify various elements needed for a collaboration diagram
4) Identify the behavior of the systems element with the external interfaces.
5) Draw the collaboration diagram and State chart diagram as per the guidelines.

Name	Eklavya Vail	Aarush Mishra	Mithas Jain
Roll no.	B117	B155	B160

Course: BTech
Branch: CE
Division: C2

1. Elements of the Sequence Diagrams

Both diagrams implement the **Boundary-Control-Entity (BCE)** architectural design pattern:

- **Actors (External Entities):**
  - Diagram 1: Patient, Caretaker / Nurse, Patient Family.
  - Diagram 2: Doctor / Dietician, Patient, Caretaker / Nurse.
- **Boundary Objects (Presentation Interfaces):**
  - PatientPortalUI: Client-side interface for patient authentication, daily vitals entry, and feedback notifications.
  - DoctorPortalUI: Clinical interface for credential intake, profile navigation, dashboard rendering, and treatment modification.
- **Controller Classes (Workflow & Business Logic):**
  - HealthMonitoringController: Manages vital parameter receipt, delegates data validation, orchestrates encrypted storage, and triggers alert pipelines.
  - ClinicalController: Authenticates clinical users, routes consent inquiries, fetches decrypted reports, and registers medication/diet revisions.
  - AccessConsentService: Validates access grants, ensuring patient medical data is accessible only with valid consent.
  - NotificationService: Dispatches push notifications and emergency text alerts across parallel threads.
- **Entity & Data Objects (Storage & Core Logic):**
  - VitalsLog: Enforces boundary logic and checks health vitals against predetermined clinical safety thresholds.
  - DatabaseService / HealthRecordService: Manages encrypted cloud data records and persistent audit trail logs.
  - DocumentVault: Encrypted document store holding patient diagnostic files and lab reports.
- **Messages:**
  - Synchronous Calls (Solid line, filled arrow): e.g., enterVitals(), submitVitals(), logVitals(), validateInputData(), saveRecord(), evaluateAnomaly(), updateMedication(), logAuditTrail(), verifyAccessConsent(), getHealthMetrics(), downloadDocument(), decryptDocument(), saveTreatmentPlan().
  - Asynchronous Signals (Solid line, stick arrow): e.g., sendCriticalAlert(), sendEmergencyNotification(), pushNotification().
  - Return Messages (Dashed line, stick arrow): e.g., validationFailed(), validationSuccess(), recordPersisted(), anomalyStatus(), alertsDispatched(), auditSaved(), consentGranted(), returnEncryptedFile().
- **Combined Interaction Fragments:**
  - alt (Alternative): Used for conditional execution (e.g., Invalid Numeric Boundaries vs. Valid Parameters; Safe Vitals vs. Anomaly Detected; Consent Denied vs. Consent Granted).
  - par (Parallel): Used for concurrent operations (e.g., simultaneous alert dispatch to Caretaker and Family; simultaneous notifications to Patient and Caretaker).

3. Elements Needed for a Collaboration (Communication) Diagram

While a Sequence Diagram organizes interactions along a linear timeline, a Collaboration Diagram represents the same dynamic behavioral scenario mapped across structural object links:

- Object Roles (Participants): Labeled in UML notation as objectName : ClassName:

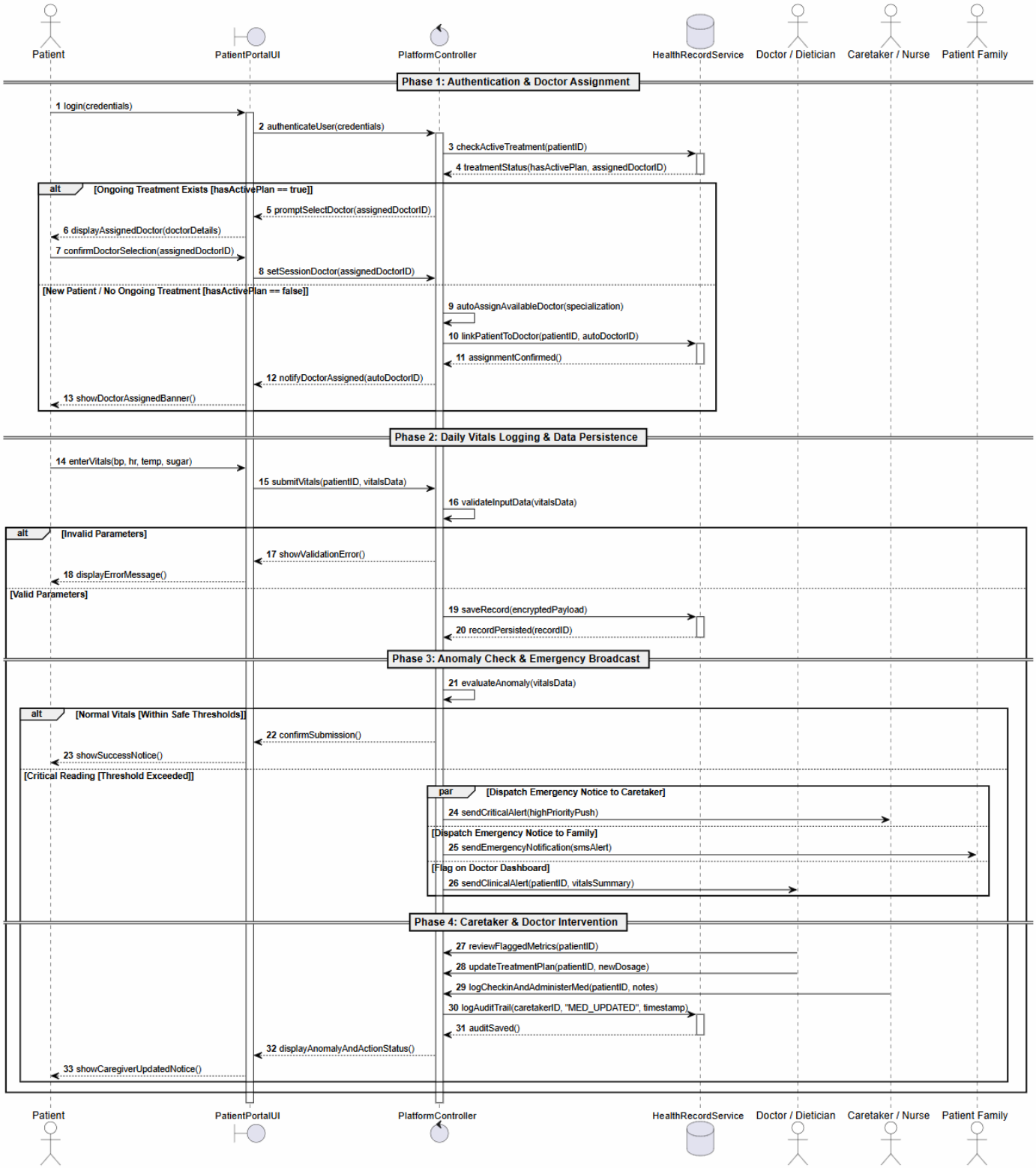
    - :PatientPortalUI, :DoctorPortalUI (Boundaries)
    - :HealthMonitoringController, :ClinicalController, :NotificationService, :AccessConsentService (Controllers)
    - :VitalsLog, :DatabaseService, :HealthRecordService, :DocumentVault (Entities / Data Stores)
- Actor Symbols: External actors (Patient, Doctor, Caretaker, Family) communicating with boundary interfaces.
- Association Paths (Links): Undirected solid lines showing structural communication channels between interacting objects.
- Hierarchically Numbered Messages: Directional arrows running parallel to the links, numbered using a nested decimal sequence to show call nesting and concurrency without lifelines:
    - 1: enterVitals(bp, hr, temp, sugar)
    - 2: submitVitals(patientID, vitalsData)
    - 2.1: logVitals(vitalsData) → 2.1.1: validateInputData()
    - 2.2: saveRecord(encryptedPayload)
    - 2.3: evaluateAnomaly()
    - 2.4 [isCritical]: triggerAlerts()
    - 2.4.1a [par]: sendCriticalAlert()
    - 2.4.1b [par]: sendEmergencyNotification()
    - 3: updateMedication(newDosage) → 3.1: logAuditTrail()

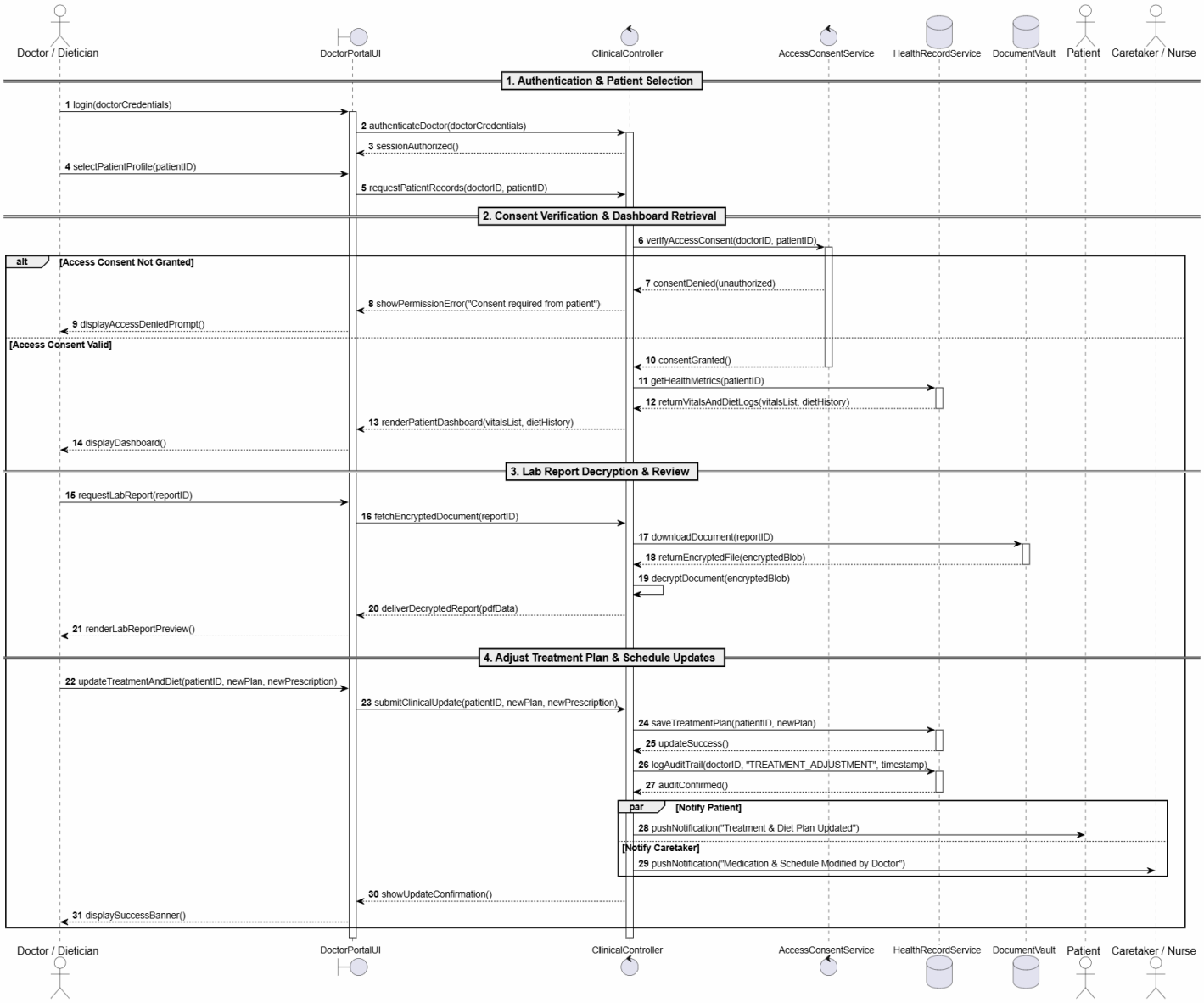
4. Behavior of System Elements with External Interfaces

The platform isolates core business logic from external boundaries through dedicated interfaces:

- Patient & Clinical Presentation Boundaries: The PatientPortalUI and DoctorPortalUI act as user-facing translation layers. They capture user actions, validate input forms client-side, delegate calls to backend controllers, and render server responses, charts, and alert dialogs.
- External Notification Gateways (Push / SMS Service): The NotificationService interfaces with third-party telecommunication and push gateways. It handles asynchronous, out-of-band broadcasts to caregiver mobile devices and family SMS endpoints without blocking primary HTTP transaction threads.
- Access Control & Consent Enforcer: The AccessConsentService functions as a security filter. It intercepts inbound doctor queries to verify explicit, active patient consent prior to retrieving or decrypting records.
- Encrypted Document & Record Vault: The DatabaseService and DocumentVault interface with the persistent cloud storage layer. They handle AES payload encryption, storage, and write-once immutable audit logs for regulatory and clinical traceability.

Output:


Patient
PatientPortalUI
PlatformController
HealthRecordService
Doctor / Dietician
Caretaker / Nurse
Patient Family
Phase 1: Authentication & Doctor Assignment
1 login(credentials)
2 authenticateUser(credentials)
3 checkActiveTreatment(patientID)
4 treatmentStatus(hasActivePlan, assignedDoctorID)
alt
[Ongoing Treatment Exists [hasActivePlan == true]]
5 promptSelectDoctor(assignedDoctorID)
6 displayAssignedDoctor(doctorDetails)
7 confirmDoctorSelection(assignedDoctorID)
8 setSessionDoctor(assignedDoctorID)
[New Patient / No Ongoing Treatment [hasActivePlan == false]]
9 autoAssignAvailableDoctor(specialization)
10 linkPatientToDoctor(patientID, autoDoctorID)
11 assignmentConfirmed()
12 notifyDoctorAssigned(autoDoctorID)
13 showDoctorAssignedBanner()
Phase 2: Daily Vitals Logging & Data Persistence
14 enterVitals(bp, hr, temp, sugar)
15 submitVitals(patientID, vitalsData)
16 validateInputData(vitalsData)
alt
[Invalid Parameters]
17 showValidationError()
18 displayErrorMessage()
[Valid Parameters]
19 saveRecord(encryptedPayload)
20 recordPersisted(recordID)
Phase 3: Anomaly Check & Emergency Broadcast
21 evaluateAnomaly(vitalsData)
alt
[Normal Vitals [Within Safe Thresholds]]
22 confirmSubmission()
23 showSuccessNotice()
[Critical Reading [Threshold Exceeded]]
par
[Dispatch Emergency Notice to Caretaker]
24 sendCriticalAlert(highPriorityPush)
[Dispatch Emergency Notice to Family]
25 sendEmergencyNotification(smsAlert)
[Flag on Doctor Dashboard]
26 sendClinicalAlert(patientID, vitalsSummary)
Phase 4: Caretaker & Doctor Intervention
27 reviewFlaggedMetrics(patientID)
28 updateTreatmentPlan(patientID, newDosage)
29 logCheckinAndAdministerMed(patientID, notes)
30 logAuditTrail(caretakerID, "MED_UPDATED", timestamp)
31 auditSaved()
32 displayAnomalyAndActionStatus()
33 showCaregiverUpdatedNotice()
Patient
PatientPortalUI
PlatformController
HealthRecordService
Doctor / Dietician
Caretaker / Nurse
Patient Family


Doctor / Dietician
DoctorPortalUI
ClinicalController
AccessConsentService
HealthRecordService
DocumentVault
Patient
Caretaker / Nurse
1. Authentication & Patient Selection
1 login(doctorCredentials)
2 authenticateDoctor(doctorCredentials)
3 sessionAuthorized()
4 selectPatientProfile(patientID)
5 requestPatientRecords(doctorID, patientID)
2. Consent Verification & Dashboard Retrieval
6 verifyAccessConsent(doctorID, patientID)
alt
[Access Consent Not Granted]
7 consentDenied(unauthorized)
8 showPermissionError("Consent required from patient")
9 displayAccessDeniedPrompt()
[Access Consent Valid]
10 consentGranted()
11 getHealthMetrics(patientID)
12 returnVitalsAndDietLogs(vitalsList, dietHistory)
13 renderPatientDashboard(vitalsList, dietHistory)
14 displayDashboard()
3. Lab Report Decryption & Review
15 requestLabReport(reportID)
16 fetchEncryptedDocument(reportID)
17 downloadDocument(reportID)
18 returnEncryptedFile(encryptedBlob)
19 decryptDocument(encryptedBlob)
20 deliverDecryptedReport(pdfData)
21 renderLabReportPreview()
4. Adjust Treatment Plan & Schedule Updates
22 updateTreatmentAndDiet(patientID, newPlan, newPrescription)
23 submitClinicalUpdate(patientID, newPlan, newPrescription)
24 saveTreatmentPlan(patientID, newPlan)
25 updateSuccess()
26 logAuditTrail(doctorID, "TREATMENT_ADJUSTMENT", timestamp)
27 auditConfirmed()
par
[Notify Patient]
28 pushNotification("Treatment & Diet Plan Updated")
[Notify Caretaker]
29 pushNotification("Medication & Schedule Modified by Doctor")
30 showUpdateConfirmation()
31 displaySuccessBanner()